



Programação Web

Javascript & Typescript

Prof. Diego Cardoso Borda Castro



Typescript

- Linguagem open-source da microsoft
- Superset do javascript
- Tipagem estática



```
1  let aluno = {
2      nome: "Diego",
3      email: "diego.castro@cefet-rj.br",
4      cpf: "11111111111"
5  }
6
7  aluno.cpf = 11111111111;
```



```
class aluno {
    nome: string;
    email: string;
    cpf: string;
}

let novo_aluno: aluno = {
    nome: "Diego",
    email: "diego.castro@cefet-rj.br",
    cpf: 11111111111
}
```



Typescript

JS

```
1 function soma(valor1, valor){
2     return valor1 + valor2;
3 }
4
5 console.log(soma(1,2));
6 console.log(soma("1","2"));
7 console.log(soma());
```

TS

```
function soma(valor1: number, valor2:number){
    return valor1 + valor2;
}

console.log(soma(1,2));
console.log(soma("1","2"));
console.log(soma());
```



Dados, variáveis e operações

- let, var ou const
- CamelCase
- snake_case

Declaração

Nome

Tipo

Valor

```
let mensagem: string = 'Teste'
```



Dados, variáveis e operações

- “Oi ” + variável + “, boa tarde!”
- `Oi \${variável}, boa tarde!`
 - Quebra de linha sem
 - “texto ” + “/n”
- typeof
- Instanceof

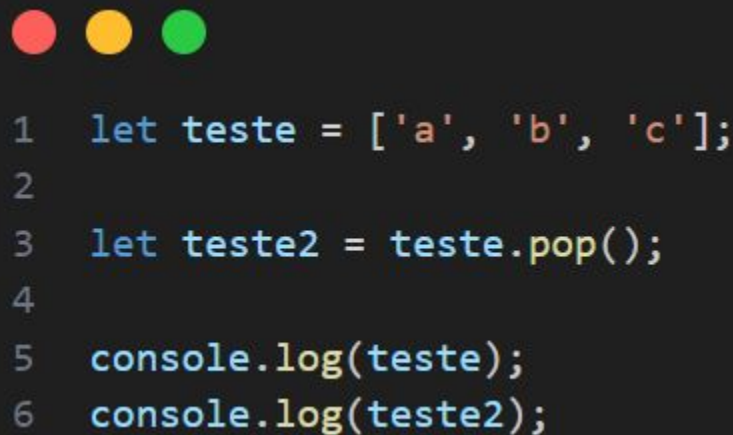
```
let example = null; // null
example = 'string'; // string
example = 3.14; // number
example = true; // boolean
example = undefined; // undefined
example = []; // array
example = Symbol("abc") // Symbol

example = { // obj
  name: 'Nicolas',
  lastName: 'Molina'
}
```



Dados, variáveis e operações

- `let numeros = [1,2,3,4,5]`
- `numeros.length`
- `numeros[0]`
- Push & Pop
- Shift
- Unshift
- `indexOf`
- `LastIndexOf`



```
1  let teste = ['a', 'b', 'c'];
2
3  let teste2 = teste.pop();
4
5  console.log(teste);
6  console.log(teste2);
```



Dados, variáveis e operações

- Slice
- Includes
- Reverse

```
1 let teste = ['a', 'b', 'c', 'd', 'e'];
2
3
4 console.log(teste.slice(2,5));
5 console.log(teste.slice(3));
```

Outros métodos

- Trim()
- Padstart(4, '0')
- Split(" ")
- ['a', 'b', 'c'].join(", ")
- Repeat(5)



Dados, variáveis e operações

- Objetos
- Copiar propriedades dos objetos
 - `Object.assign(a, b)`
- Chaves de um objeto
 - `Object.keys(a)`
- Referência
 - `carroA = CarroB`

```
1  let carroA = {
2    portas: 4,
3    marca: "XPTO",
4  }
5
6  let carroB = {
7    motor: 1.0
8  }
9
10 Object.assign(carroA, carroB)
11 console.log(Object.keys(carroA));
12 console.log(carroB);
```




Dados, variáveis e operações

- Objetos
- Copiar propriedades dos objetos
 - `Object.assign(a, b)`
- Chaves de um objeto
 - `Object.keys(a)`
- Referência
 - `carroA = CarroB`

```
1  let carroA = {
2      portas: 4,
3      marca: "XPTO",
4  }
5
6  let carroB = {
7      motor: 1.0
8  }
9
10 carroA = carroB;
11 carroB.motor = 2.0;
12 console.log(carroA);
```



Operadores Aritméticos

- Símbolos especiais que envolvem um ou mais operandos
- NaN
 - Not a number
- +=

Descrição	Operador	Exemplo
ADIÇÃO	+	2 + 8
SUBTRAÇÃO	-	2 - 8
MULTIPLICAÇÃO	*	2 * 8
DIVISÃO	/	2 / 8
EXPONENCIAÇÃO (OU POTENCIALIZAÇÃO)	**	2 ** 8
RESTO DA DIVISÃO (OU MODULUS)	%	2 % 8
INCREMENTO	++	1++
DECREMENTO	--	1--



Operadores Booleanos

- ==
- >
- <
- >=
- <=
- !=
- ===

Operador	Nome da Operação	Descrição
&&	AND (operação e)	Usamos essa operação quando desejamos que DUAS condições sejam SIMULTANEAMENTE verdadeiras.
	OR (operação ou)	Usamos essa operação quando desejamos que pelo menos UMA das condições seja verdadeira.
!	NOT (Inversor)	Esse é um operador aplicado a UM VALOR booleano quando queremos o seu INVERSO (NÃO É algo).



Destructuring

- Desconstruir objetos
- Interface
- Passagem de parametros

```
const person = {  
  name: 'Jhon',  
  lastname: 'Doe'  
}  
  
const {name: fname, lastname: lname} = person;
```

```
let nomes = ['Matheus', 'João', 'Pedro'];  
  
let [nomeA, nomeB, nomeC] = nomes;  
  
console.log(nomeA);  
console.log(nomeB);  
console.log(nomeC);
```



Funções

- Reutilização
- Parâmetros
- Export
- Arrow function

```
1 function soma(valor1, valor){
2     return valor1 + valor2;
3 }
4
5 console.log(soma(1,2));
```

```
const myFunc = () => {
    //do something
}

const myFunc = (arg1, arg2) => {
    //do something
}

const myFunc = args => dosomething()

const myFunc = () => dosomething()
```



Funções

- Reutilização
- Parâmetros
- Export
- Arrow function

```
1  let myFunction = (a, b) => a * b;  
2  console.log(myFunction(2,5));  
3  
4  let y = x => x * 2;  
5  console.log(y(4));  
6  
7  let isPositive = number => number >= 0;  
8  console.log(isPositive(5));  
9
```



Funções

- Valores default
- Valores opcionais
- Closure
 - Se lembra do ambiente que foi criada
- Recursividade

```
1 function soma(a){  
2     return y => y + a  
3 }  
4  
5 let cont = soma(3)  
6 console.log(cont(5));
```

```
1 let retorno = function(a: number = 2, b?: number){  
2     return a;  
3 }  
4
```



Condicionais

- if, else if, else
- Switch, case

```
switch(nome) {  
  case "João":  
    console.log("O seu nome é João");  
    break;  
  case "Marcos":  
    console.log("O seu nome é Marcos");  
    break;  
  default:  
    console.log("O seu nome não é João nem Marcos!");  
    break;  
}
```

Repetição

- While
- Do while
- For
- ForEach

```
1  const frutas = ['maçã', 'banana', 'laranja', 'uva'];  
2  frutas.forEach(function(fruta) {  
3    console.log(fruta);  
4  });
```




Prototypes

- Molde para os outros objetos
- Classe

```
1  let cachorro = {  
2    raca: 'Bulldogue'  
3  }  
4  
5  let pastor = Object.create(cachorro);  
6  
7  console.log(pastor.raca);  
8  console.log(Object.getPrototypeOf(pastor) == cachorro);  
9  
10 pastor.raca = 'Pastor Alemão';  
11 console.log(pastor.raca);  
12 console.log(cachorro.raca);  
13
```



Construtores

- Prototypes
- Objetos
- New
- Getters e setters
- Extends

```
1  class Cachorro {
2      constructor(raca){
3          this.raca = raca;
4      }
5
6      latir(){
7          return 'au au';
8      }
9  }
10
11  Cachorro.prototype.patas = 4;
12
13  let poodle = new Cachorro('Poodle');
14  console.log(poodle);
15  console.log(poodle.prototype.patas);
16  console.log(Cachorro.prototype)
```



Construtores

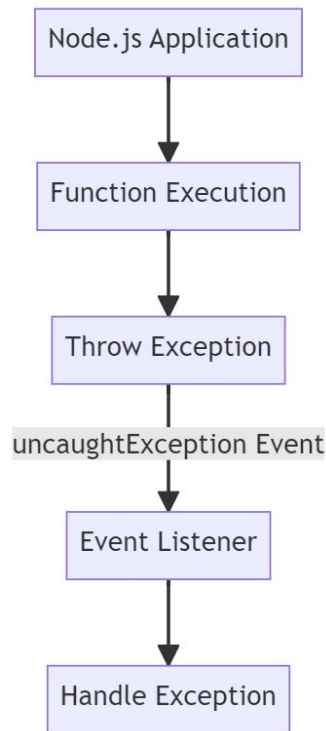
- Prototypes
- Objetos
- New
- Getters e setters
- Extends

```
1  class Mamifero {  
2      constructor(patas){  
3          this.patas = patas;  
4      }  
5  }  
6  
7  class Cachorro extends Mamifero {  
8      constructor(raca, patas){  
9          super(patas);  
10         this.raca = raca  
11     }  
12 }  
13  
14 let poodle = new Cachorro('Poodle', 4);  
15 console.log(poodle);
```



Exceptions

- ErrorHandler
 - Método geral
 - Mensagens padronizadas
 - Único ponto de tratativa
- Throw new Error
- Try, Catch, Finally





Programação assíncrona

- Callback
 - Função que faz uma ação após algum acontecimento no código;

```
$('#MinhaDiv').hide(3000, mensagem());  
  
function mensagem(){  
    alert('A minha Div Sumiu!!');
```

```
function oddOrEven(number, callback) {  
    const result = (number % 2 == 0) ? 'Even' : 'Odd';  
    callback(number, result);  
}  
  
oddOrEven(28, (number, result) => {  
    console.log(number + ' is ' + result);  
});  
  
// 28 is Even
```



Programação assíncrona

- Promises
 - then
- Promise.all
- Banco de dados

```
1 let promessa = Promise.resolve(4 + 8);
2
3 console.log('Esperando...');
4
5 promessa.then((x) => {
6     console.log(`valor: ${x}`)
7 }).catch( erro => console.log(error))
```

```
let promessa = new Promise((resolve, reject) => {
    if(...)
        resolve(console.log('...'))
    else
        reject(new Error('Error'));
})
```



Programação assíncrona

- Promises
 - then
- Promise.all
- Banco de dados

```
function somaComDelay(a,b) {  
  return new Promise(resolve => {  
    setTimeout(function() {  
      resolve(a + b);  
    }, 2000);  
  });  
}  
  
async function soma(a,b,c,d) {  
  let x = somaComDelay(a,b);  
  let y = somaComDelay(c,d)  
  
  return await x + await y;  
}
```



Spread Vs Rest

- ...
- array ou string seja expandido

```
1 const array1 = [1,2, 3]
2 const array2 = [4,5, 6]
3
4 const merge = [
5   ...array1,
6   ...array2
7 ]
8
9 console.log(merge)
```

```
1 function soma(...args){
2   return args.reduce((a,b) => a + b);
3 }
4
5 console.log(soma(1,2,3,4,5,6))
```




Programação funcional

- Tudo pode ser uma função
 - Church encoding
- Paradigma de programação
- Funções matemática
- Funções como funções
- Imutável
- Stateless
- Independente
- Sem loop
- Recursão
- **Programação interativa**
- Sem efeito externo



Programação funcional

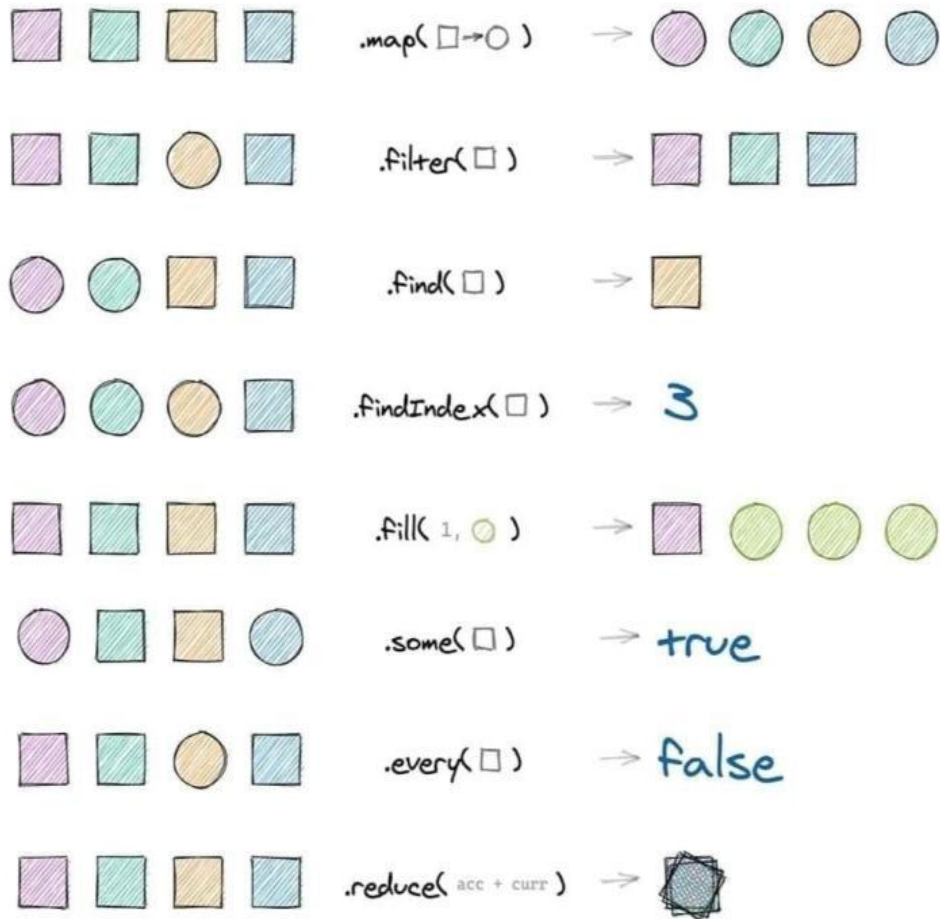
- Abstração
- Função anônima
- Map
 - Lista em nova lista

```
1  const array1 = [5, 12, 8, 130, 44];
2
3  const found = array1.find((element) => element > 10);
4
5  function maior10(el, index, array){
6      return el > 10;
7  }
8
9  console.log(array1.find(maior10));
10 console.log(array1.filter(maior10));
11 console.log(found)
```



Programação funcional

- Função anônima
- Map
 - Lista em nova lista





Interfaces

- Contrato que define o que o objeto deve fazer
 - Mas não como
 - Nome do método e os seus parâmetros
 - Contrato de implementação



Últimas dicas

- Extensão Prettier - Code formatter
 - Alt + shift + F
- Comentário
 - // Comentário de linha
 - /* */ Bloco de comentário
- Code runner

